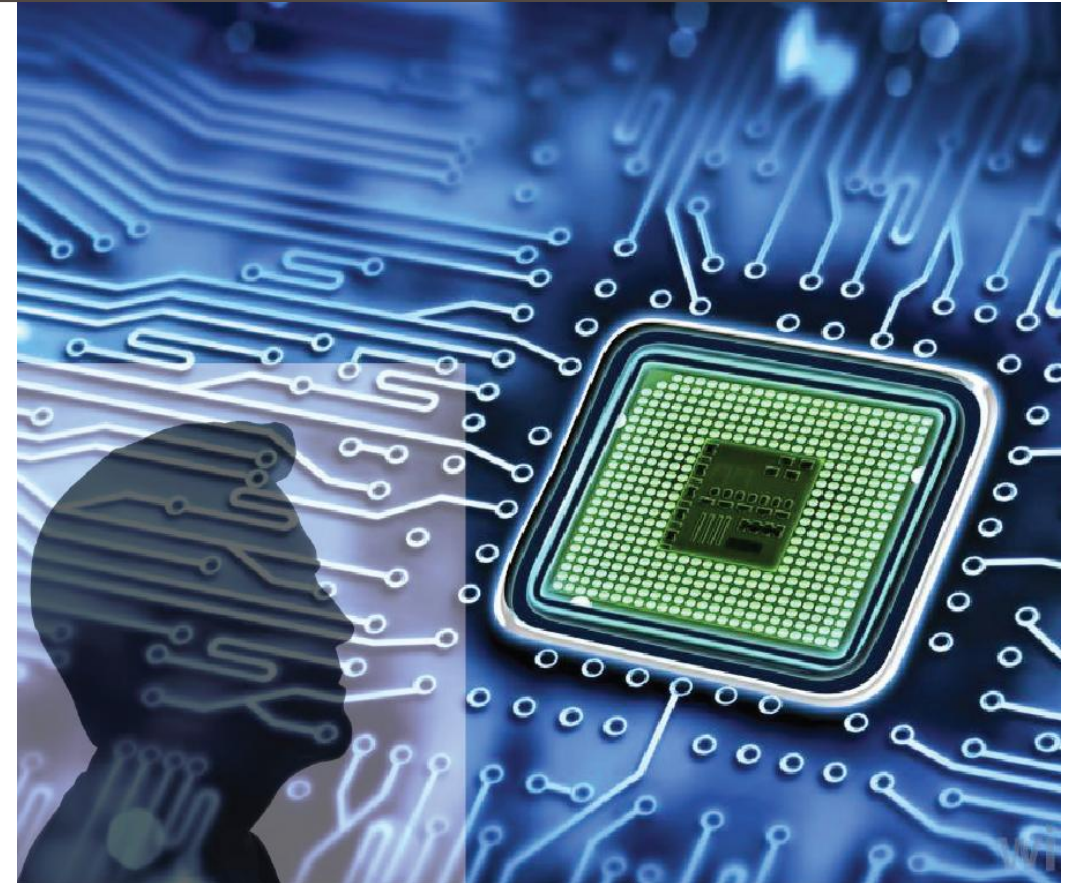


# ADVANCED DIGITAL DESIGN AND VERIFICATION

## Week-13 Lecture-2 UVM

Sneh Saurabh  
20<sup>th</sup> November 2025



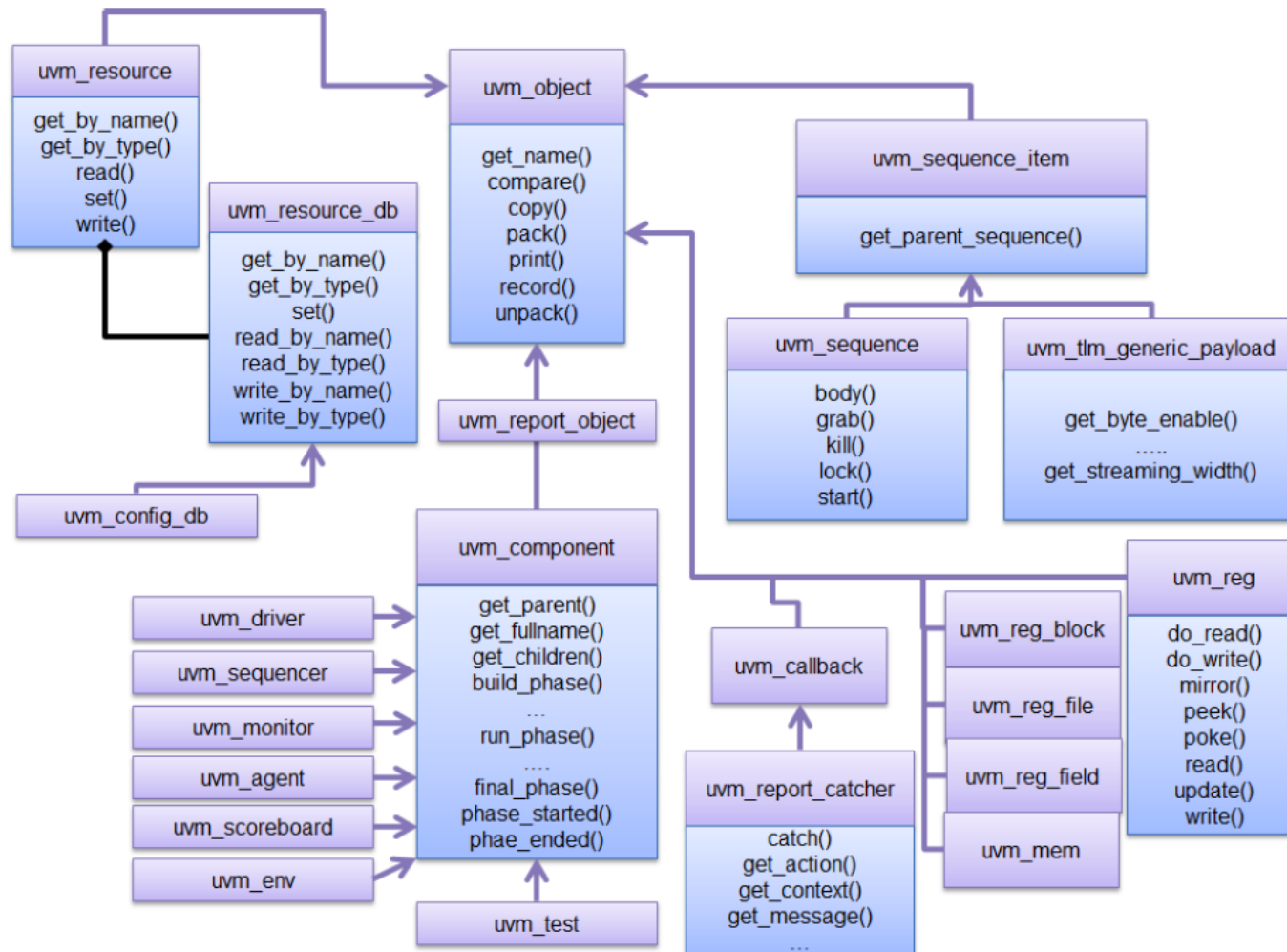
---

---

# UVM: Classes

# UVM: Class Libraries (1)

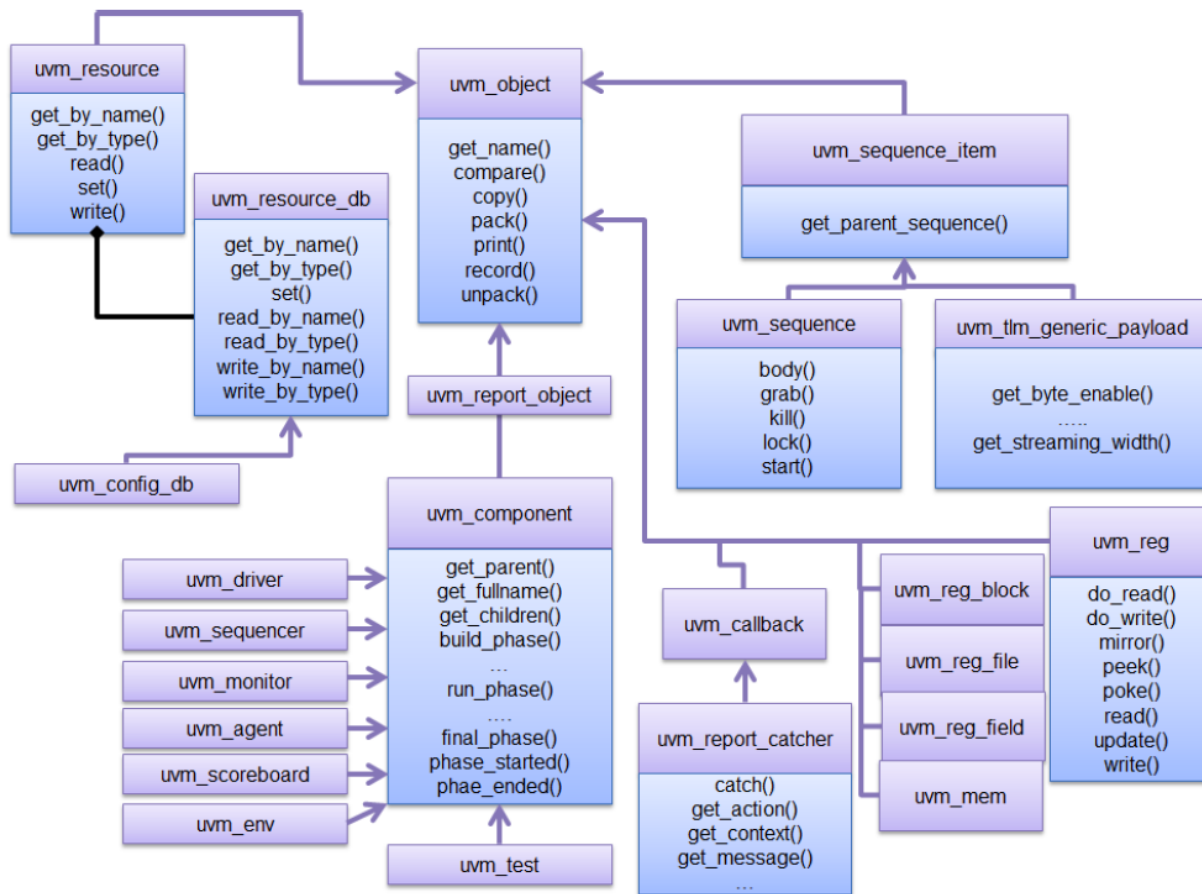
UVM Class Library provides building blocks to quickly develop test environments



## Base Class

- **uvm\_object**: root class of all UVM objects
- Provides common methods: **get\_name()**, **copy()**, **compare()**, **print()**, **pack()**, **unpack()** etc.

# UVM: Class Libraries (2)



## For Building Architecture

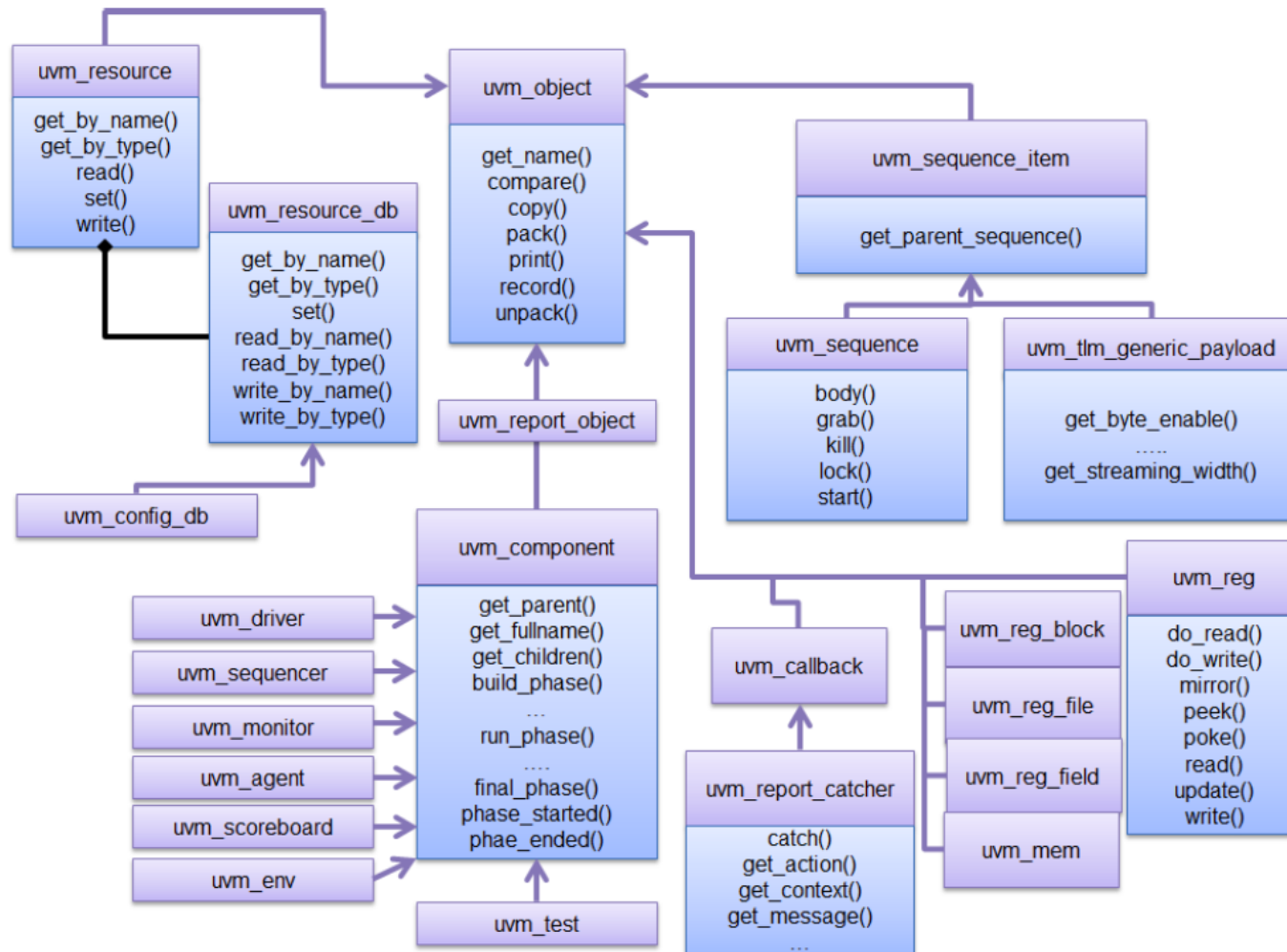
- Main architectural components are derived from **uvm\_component**

- **uvm\_test**
- **uvm\_env**
- **uvm\_scorecard**
- **uvm\_agent**
- **uvm\_monitor**
- **uvm\_sequencer**
- **uvm\_driver**

- **uvm\_component** is a hierarchical elements with phasing

- We can derive components from the UVM Class Library components in UVM-based testing: reusable, saves effort, readability

# UVM: Class Libraries (3)

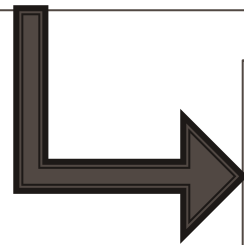


Data classes:

- **uvm\_sequence**: generate and control stimulus for verification (defines the pattern of transactions)
- **uvm\_tlm\_generic\_payload**: standardized transaction format for bus operations
- **uvm\_resource\_db**: global storage mechanism in UVM that allows sharing of data and objects across different components connections.
- Utilities (such as for reporting, debugging)

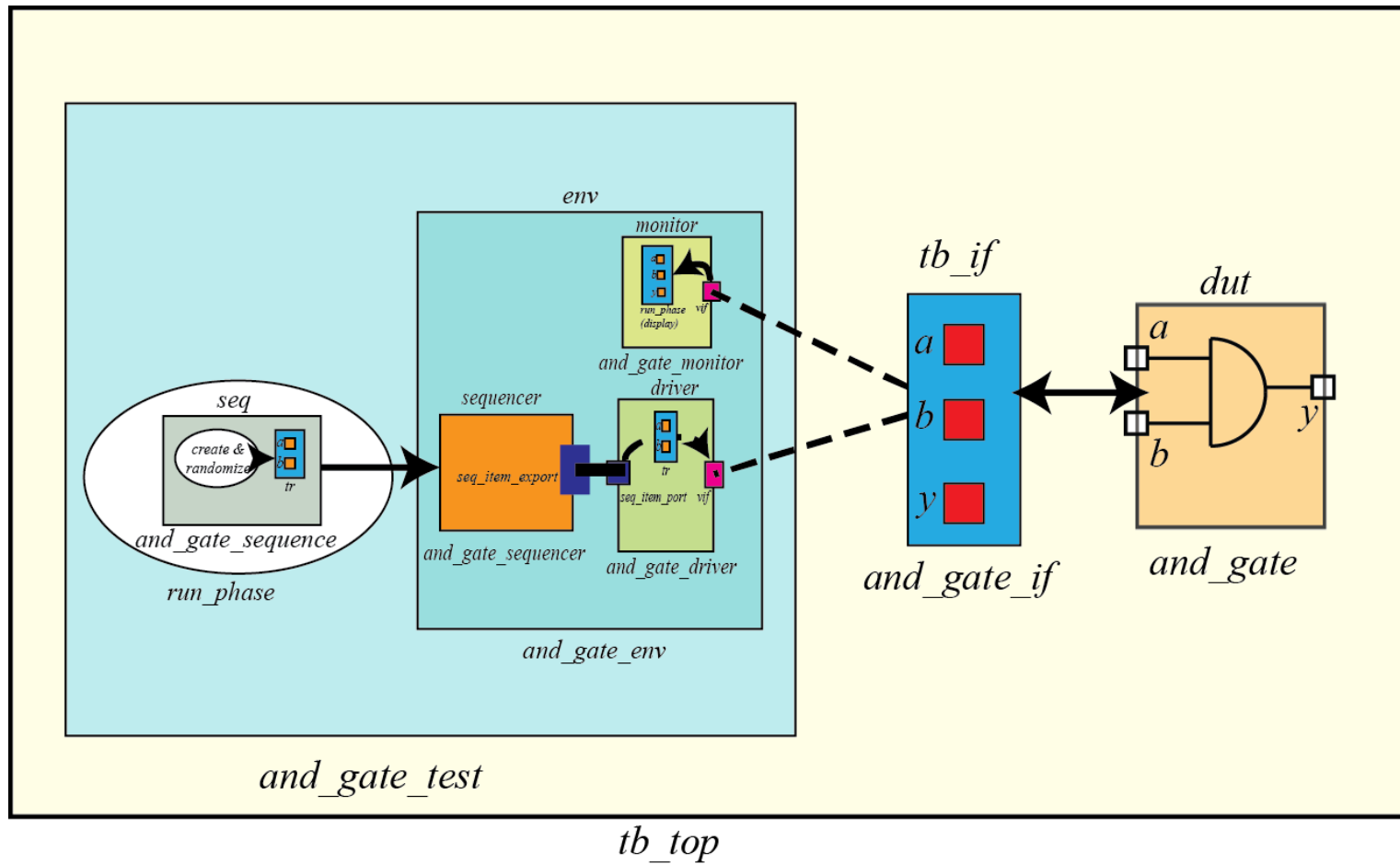
---

# Universal Verification Methodology (UVM)



## Example Testbench

# UVM: Example (Overview)



# UVM: Running Verilog Simulator (NCSim)

```
[sneh@edatools-server1 uvm-1]$ ncoverilog -uvm -uvmhome CDNS-1.1d -f file_list.f  
+UVM_TESTNAME=and_gate_test
```

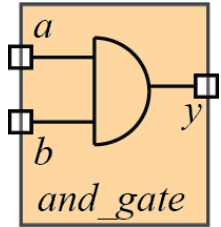
```
[sneh@edatools-server1 uvm-1]$ ls /cadence/INCISIVE152/tools/methodology/UVM/CDNS-1.1d/sv/src/base/  
uvm_barrier.svh          uvm_event.svh          uvm_pool.svh           uvm_resource_specializations.svh  
uvm_base.svh            uvm_factory.svh        uvm_port_base.svh      uvm_resource.svh  
uvm_bottomup_phase.svh  uvm_globals.svh        uvm_printer.svh        uvm_root.svh  
uvm_callback.svh        uvm_heartbeat.svh      uvm_queue.svh          uvm_runtime_phases.svh  
uvm_cmdline_processor.svh uvm_if_base.svh        uvm_recorder.svh       uvm_spell_chkr.svh  
uvm_common_phases.svh  uvm_misc.svh           uvm_registry.svh       uvm_task_phase.svh  
uvm_comparer.svh        uvm_object_globals.svh uvm_report_catcher.svh uvm_topdown_phase.svh  
uvm_component.svh      uvm_objection.svh      uvm_report_handler.svh uvm_transaction.svh  
uvm_config_db.svh      uvm_object.svh         uvm_report_object.svh  uvm_version.svh  
uvm_domain.svh         uvm_packer.svh         uvm_report_server.svh  
uvm_event_callback.svh uvm_phase.svh          uvm_resource_db.svh
```

file\_list.f

```
and_gate.sv  
and_gate_if.sv  
and_gate_transaction.sv  
and_gate_driver.sv  
and_gate_monitor.sv  
and_gate_env.sv  
and_gate_sequence.sv  
and_gate_test.sv  
tb_top.sv
```

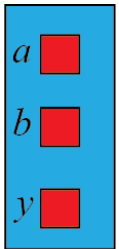
# UVM: DUT, Interface and Transaction

and\_gate.sv



```
module and_gate (  
    input logic a,  
    input logic b,  
    output logic y  
);  
    assign y = a & b;  
endmodule
```

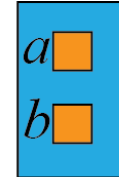
and\_gate\_if.sv



*and\_gate\_if*

```
interface and_gate_if;  
    logic a;  
    logic b;  
    logic y;  
endinterface
```

and\_gate\_transaction.sv

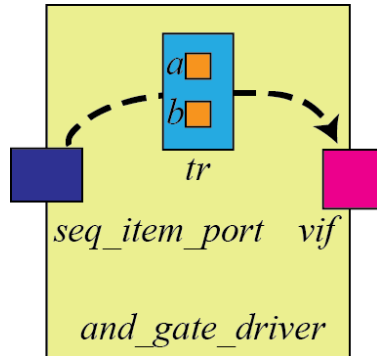


*and\_gate\_transaction*

```
`include "uvm_macros.svh"  
import uvm_pkg::*;  
  
class and_gate_transaction extends uvm_sequence_item;  
  
    rand logic a, b;  
  
    `uvm_object_utils(and_gate_transaction)  
  
    function new(string name = "and_gate_transaction");  
        super.new(name);  
    endfunction  
  
    function void display();  
        $display("TRANSACTION: a=%0b, b=%0b", a, b);  
    endfunction  
  
endclass
```

# UVM: Driver

and\_gate\_driver.sv



```

44 class uvm_driver #(type REQ=uvm_sequence_item,
45                     type RSP=REQ) extends uvm_component;
46
47 // Port: seq_item_port
48 //
49 // Derived driver classes should use this port to
50 // request items from the
51 // sequencer. They may also use it to send responses
52 // back.
53 uvm_seq_item_pull_port #(REQ, RSP) seq_item_port;

```

```

class and_gate_driver extends uvm_driver #(and_gate_transaction);
    virtual and_gate_if vif;

    `uvm_component_utils(and_gate_driver)

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual and_gate_if)::get(this, "", "vif", vif))
            `uvm_fatal("NOVIF", "Virtual interface not set")
    endfunction

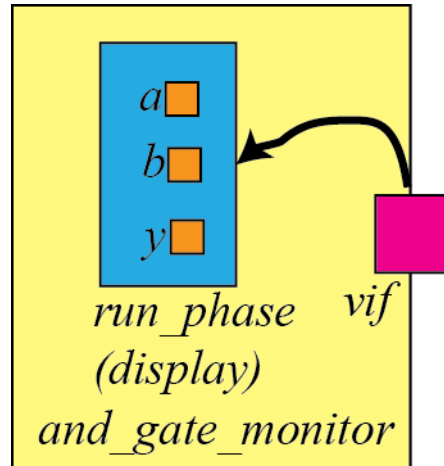
    task run_phase(uvm_phase phase);
        and_gate_transaction tr;
        forever begin
            seq_item_port.get_next_item(tr);
            vif.a = tr.a;
            vif.b = tr.b;
            seq_item_port.item_done();
        end
    endtask
endclass

```

- Driver pulls transaction
- `get_next_item()` is blocking (waits until it gets one)
- `item_done()` informs sequencer that it is done with processing (could have sent response also)

# UVM: Monitor

and\_gate\_monitor.sv



```
class and_gate_monitor extends uvm_monitor;
    virtual and_gate_if vif;

    `uvm_component_utils(and_gate_monitor)

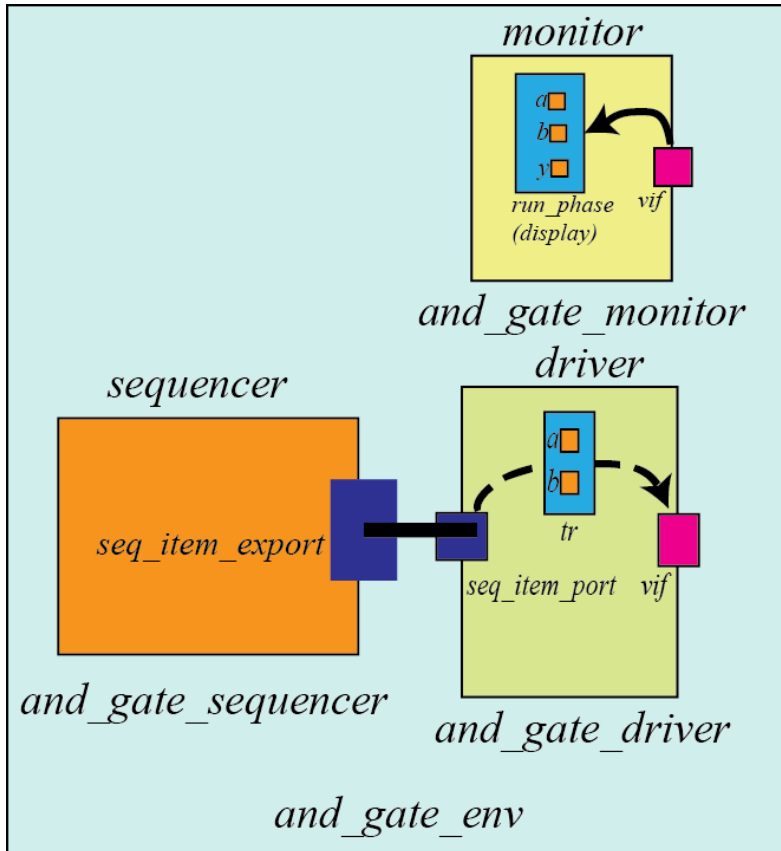
    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual and_gate_if)::get(this, "", "vif", vif))
            `uvm_fatal("NOVIF", "Virtual interface not set")
        endfunction

    task run_phase(uvm_phase phase);
        forever begin
            @(vif.a or vif.b or vif.y);
            $display("MONITOR: a=%0b, b=%0b, y=%0b", vif.a, vif.b, vif.y);
        end
    endtask
endclass
```

# UVM: Environment

and\_gate\_env.sv



```
class and_gate_env extends uvm_env;
    and_gate_driver driver;
    and_gate_monitor monitor;

    uvm_sequencer #(and_gate_transaction) sequencer;

    `uvm_component_utils(and_gate_env)

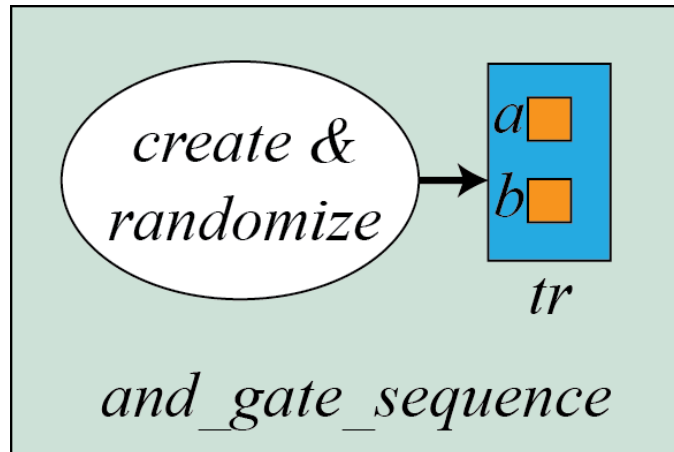
    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        driver = and_gate_driver::type_id::create("driver", this);
        monitor = and_gate_monitor::type_id::create("monitor", this);
        sequencer =
            uvm_sequencer #(and_gate_transaction)::type_id::create("sequencer", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        driver.seq_item_port.connect(sequencer.seq_item_export);
    endfunction
endclass
```

# UVM: Sequence

and\_gate\_sequence.sv



## start\_item()

- Requests grant from the sequencer
- Waits until driver is ready to accept the item (driver calls `get_next_item()`)

```
class and_gate_sequence extends uvm_sequence #(and_gate_transaction);  
    `uvm_object_utils(and_gate_sequence)  
  
    function new(string name = "and_gate_sequence");  
        super.new(name);  
    endfunction  
  
    task body();  
        and_gate_transaction tr;  
        repeat (6) begin  
            tr = and_gate_transaction::type_id::create("tr");  
            start_item(tr);  
            assert(tr.randomize());  
            tr.display();  
            finish_item(tr);  
        end  
    endtask  
endclass
```

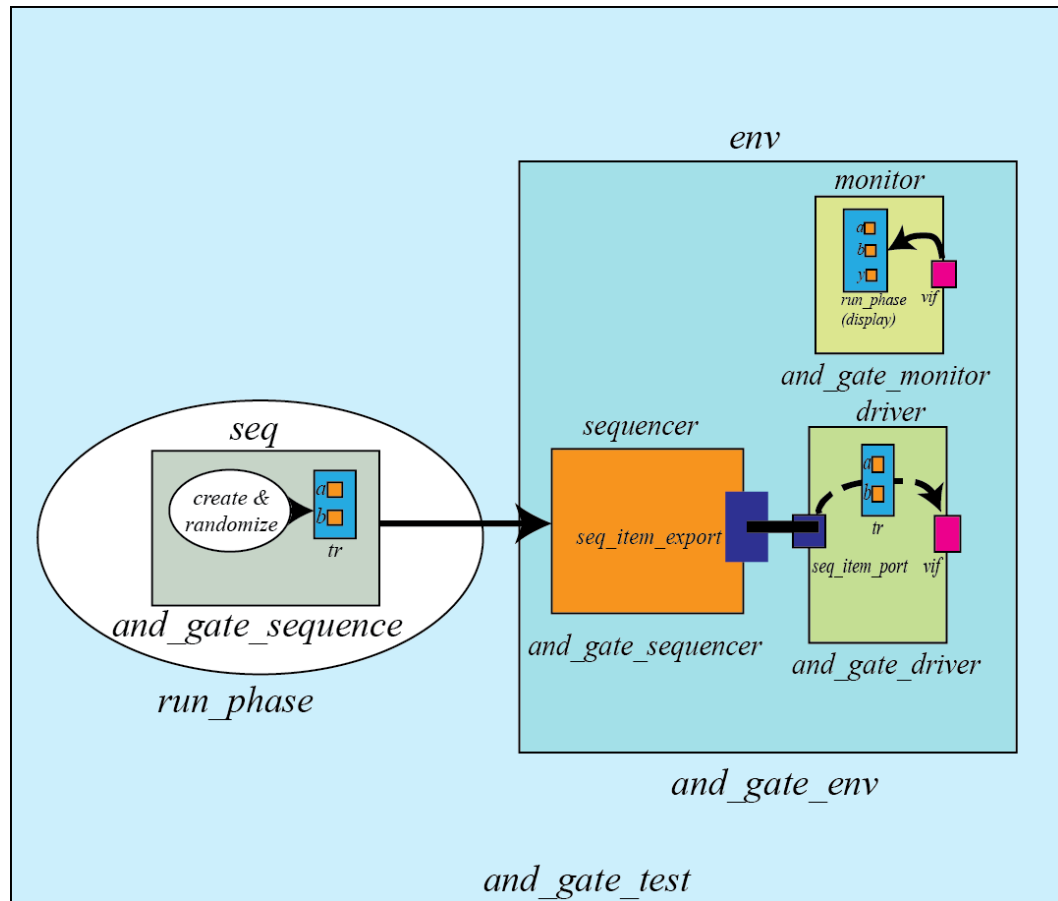
- Blocking transactions

## finish\_item()

- Blocks until driver calls `item_done()`

# UVM: Test

and\_gate\_test.sv



```
class and_gate_test extends uvm_test;
    and_gate_env env;

    `uvm_component_utils(and_gate_test)

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        env = and_gate_env::type_id::create("env", this);
    endfunction

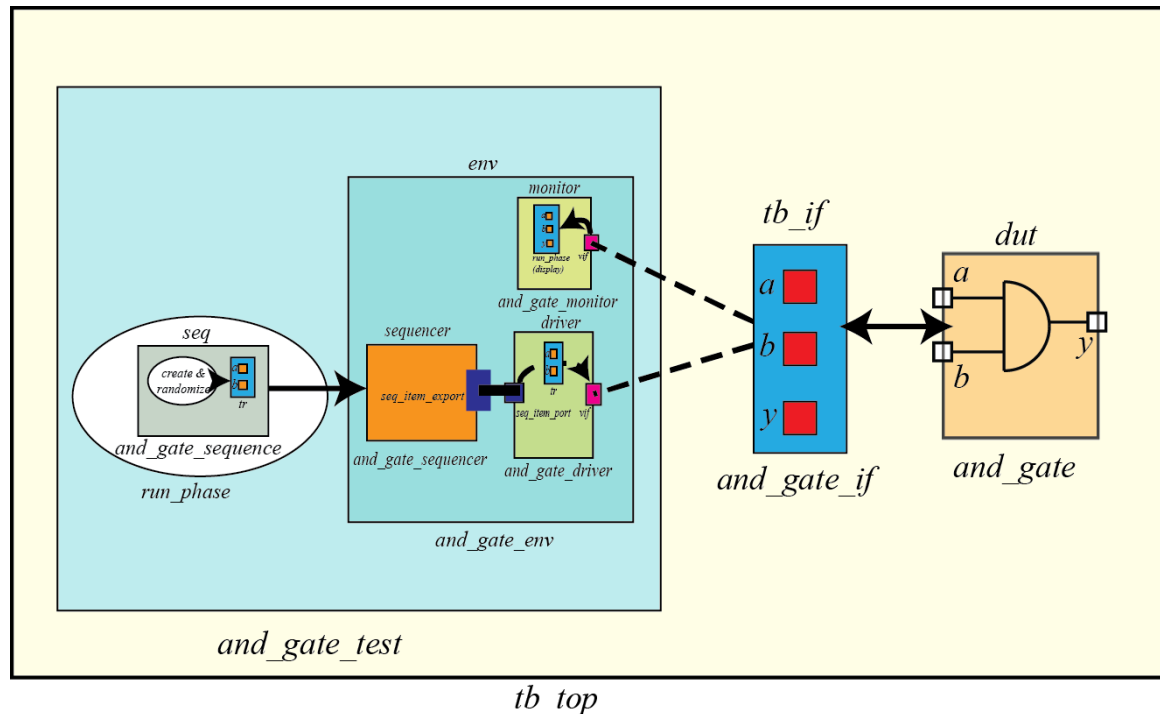
    task run_phase(uvm_phase phase);
        and_gate_sequence seq;
        seq = and_gate_sequence::type_id::create("seq");
        seq.start(env.sequencer);
    endtask
endclass
```

## seq.start()

- Gives the context to the sequence (using `env.sequencer` driver linked to sequence)
- Internally calls `body()` of sequence

# UVM: Testbench Top

tb\_top.sv



```
module tb_top;
    and_gate_if tb_if();
    and_gate dut (
        .a(tb_if.a),
        .b(tb_if.b),
        .y(tb_if.y)
    );

    initial begin
        uvm_config_db#(virtual and_gate_if)::set(null,
            "*", "vif", tb_if);
        run_test("and_gate_test");
    end
endmodule
```

- Actual interface created here:  
*tb\_if()*
- The created interface is passed to UVM config db (using **set**(..."vif"..))
- When other components (driver and monitor) query using **get**(..."vif"), they will get the handle to this created interface

# UVM: Output of NCSim

---

```
[sneh@edatools-server1 uvm-1]$ ncverilog -uvm -uvmhome CDNS-1.1d -f file_list.f  
+UVM_TESTNAME=and_gate_test
```

```
UVM_INFO @ 0: reporter [RNTST] Running test and_gate_test...  
TRANSACTION: a=1, b=1  
MONITOR: a=1, b=1, y=x  
MONITOR: a=1, b=1, y=1  
TRANSACTION: a=1, b=1  
TRANSACTION: a=1, b=1  
TRANSACTION: a=1, b=1  
TRANSACTION: a=1, b=0  
MONITOR: a=1, b=0, y=1  
MONITOR: a=1, b=0, y=0  
TRANSACTION: a=1, b=1  
MONITOR: a=1, b=1, y=0  
MONITOR: a=1, b=1, y=1
```

```
--- UVM Report catcher Summary ---
```

```
Number of demoted UVM_FATAL reports : 0  
Number of demoted UVM_ERROR reports : 0  
Number of demoted UVM_WARNING reports: 0  
Number of caught UVM_FATAL reports : 0  
Number of caught UVM_ERROR reports : 0  
Number of caught UVM_WARNING reports : 0
```

```
--- UVM Report Summary ---
```

```
** Report counts by severity
```

```
UVM_INFO : 1  
UVM_WARNING : 0  
UVM_ERROR : 0  
UVM_FATAL : 0  
... ..
```

---

---

# Portable Test and Stimulus Standard (PSS)

# Portable Test and Stimulus Standard (PSS)

---

- Portable Test and Stimulus Standard (PSS) is used to model **verification intent**
- Portable Stimulus is a verification language defined by **Accellera**
  - First version 1.0 was released June 2018
  - Latest Version 3.0 released in August, 2024
- Can be downloaded:  
<https://www.accellera.org/downloads/standards/portable-stimulus>



Portable Test and Stimulus Standard  
Version 3.0

August 2024

PSS defines a specification for:

- Creating a **single representation** of stimulus and test scenarios
- Usable across different levels of integration under **different configurations**
- Enabling the **generation of different implementations** of a scenario that run on a variety of execution platforms:
  - Simulation
  - Emulation
  - Post-silicon, and ...

# PSS: Languages and Model

---

Two input languages are defined.

1. Domain specific language (DSL): similar to existing verification languages, such as SystemVerilog
2. C++ coupled with a runtime library: unlike SystemVerilog and C++, the language is primarily used in a declarative manner rather than procedural

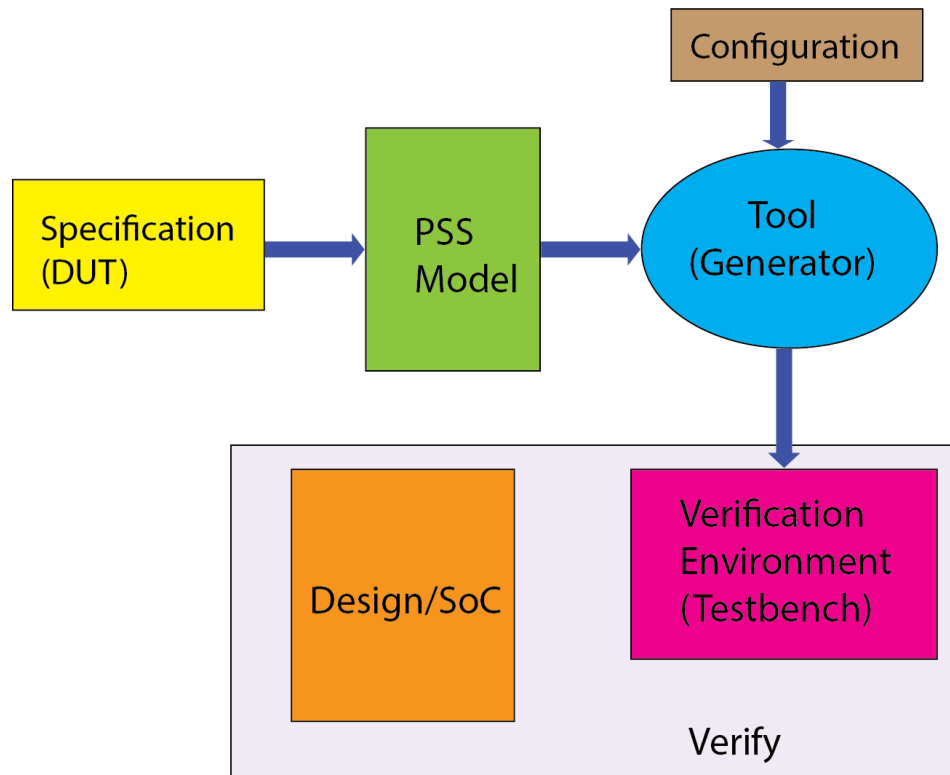
- Both languages define the same semantics.
- The DSL and C++ may be freely intermixed and should be portable between vendors.

## Model

- Using PSS a model is created and not an executable (unlike traditional C++).
  - Model is similar to a Verilog design description
- Model needs a simulator or synthesis engine to read it and to perform an action on it.

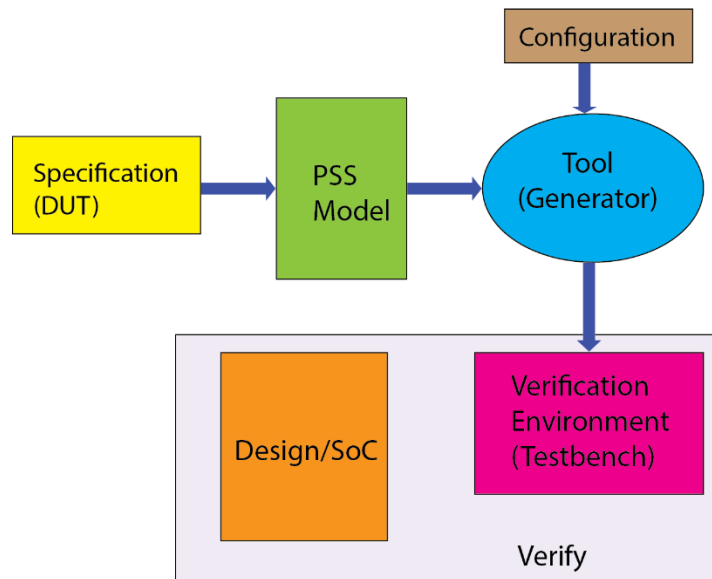
# PSS: Usage and Flow

---



- PSS model is synthesized into a testbench targeting either simulation or emulation.
- Many other verification tasks could be targeted by a synthesis engine.

# PSS: Developing Model



Developing models involves two main tasks:

1. Reusable Behavior
2. Project-specific Use Cases

## Reusable Behavior

What the hardware can do?

Model the system behaviors and their composition rules.

Example: memory\_read, memory\_write, DMA\_transfer

Reused across projects and in subsystem context

## Project-specific Use Cases

How we test specific features?

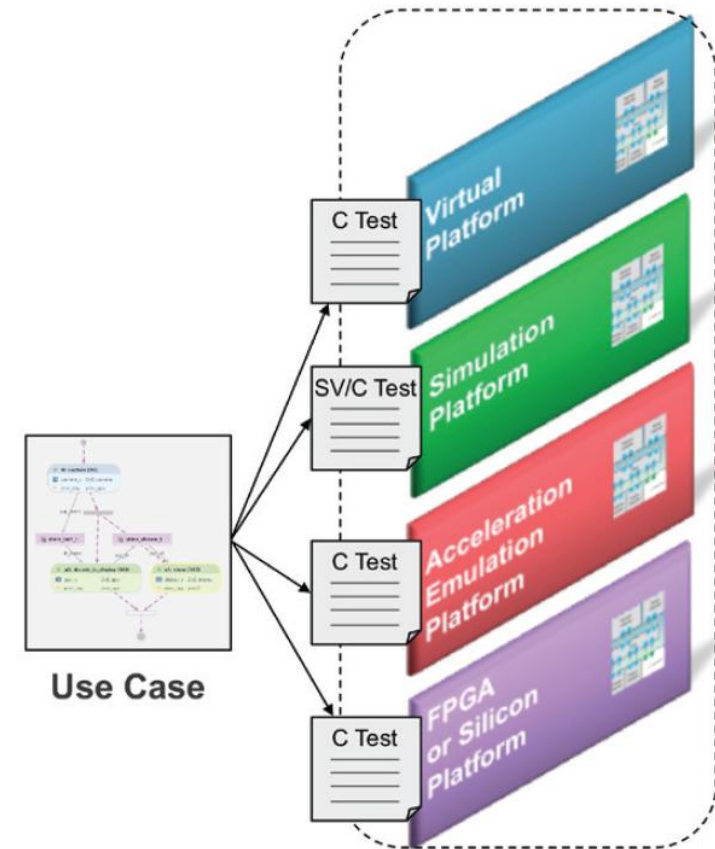
Compose use cases with constraints on control flow, data flow, and scheduling

Example: Cache-coherence test, state\_transition\_test

Project-specific scenarios considered

# PSS: Portability

- PSS allows executing the same test intent in different target platforms
  - Teach the tool more than a single API style
  - Ask the tool to realize tests in the required target platform



“Three Things You Need to Know to Use the Accellera PSS”, [https://www.cadence.com/content/dam/cadence-www/global/en\\_US/documents/tools/system-design-verification/accellera-pss-wp.pdf](https://www.cadence.com/content/dam/cadence-www/global/en_US/documents/tools/system-design-verification/accellera-pss-wp.pdf)

# PSS: Actions and Activity

- The standard is based on the mathematical notions of **graphs**
- Graph contains:
  - Atomic actions
  - Data each action requires or generates

- **Action:** defines the behavior or operation to be performed
- Actions may be composable into more complex actions

- **Activity:** specify sequences of actions or parallel operations to form complex **scenarios**.

- Other elements: constraints, resources and dependencies, coverage

```
class cpu_c : public component {
public:
    PSS_CTOR(cpu_c, component);

    class write_data : public action {
    public:
        PSS_CTOR(write_data, action);

        output<data_buff_s> dst {"dst"};
        lock<core_s> core {"core"};
    };
    type_decl<write_data> write_data_decl;

    class read_data : public action {
    public:
        PSS_CTOR(read_data, action);

        input<data_buff_s> src {"src"};
        lock<core_s> core {"core"};
    };
    type_decl<read_data> read_data_decl;
};
```

```
activity a {
    sequence {
        wd, // initialize buffer by a random core
        repeat { count,
            parallel { // do parallel transfers
                xfer1,
                xfer2
            }
        }
    }
};
```

# PSS: Notion of Stimulus, Coverage, and Checkers

---

## Stimulus and scenarios

- The graph defines complete paths through a design.
- A synthesis engine can pick random paths through the graph obtaining a complete scenarios

## Coverage

- If all paths have been exercised, then all defined actions have been involved in verification
- This notion of coverage is known prior to a test actually being run.

## Checker

- Graph actions can define their expected outcomes which can be checked during design execution

PSS is unlike SystemVerilog and UVM which have separate models for each verification component: stimulus, checkers and coverage.

---

---

# Scenario-based Software-driven Testing

# Scenario-based Software-driven Testing (SBSDT)

---

- Scenario-based Software-driven Testing (SBSDT) **uses real-world scenarios and software** to validate the functionality, performance, and reliability of SoCs
- Focus is on higher-level, software-driven scenarios that closely **mimic end-user or system-level use cases**
  - Examples: Booting an OS, streaming video, performing encryption, or data transfer between peripherals
- Executed using software that interacts with the hardware (via drivers, APIs, or firmware)
- Ensures the system behaves as expected in real-world conditions.

# References

---

- Universal Verification Methodology (UVM) 1.2 User's Guide  
[https://www.accellera.org/images/downloads/standards/uvm/uvm\\_users\\_guide\\_1.2.pdf](https://www.accellera.org/images/downloads/standards/uvm/uvm_users_guide_1.2.pdf)
- “Portable Test and Stimulus 3.0” <https://www.accellera.org/downloads/standards/portable-stimulus>
- “Three Things You Need to Know to Use the Accellera PSS”,  
[https://www.cadence.com/content/dam/cadence-www/global/en\\_US/documents/tools/system-design-verification/accellera-pss-wp.pdf](https://www.cadence.com/content/dam/cadence-www/global/en_US/documents/tools/system-design-verification/accellera-pss-wp.pdf)
- Saurabh, Sneha. Introduction to VLSI design flow. Cambridge University Press, 2023.